

0.1 Protocol Motivation

UltraGroth is addressed to insignificantly modify Groth16 protocol to enable sampling random field elements effectively and further pass them as signals to the circuit. The primary implication of such possibility is, of course, lookup checks!

Moreover, such modification saves all the verification benefits of Groth16: in UltraGroth's basic form, it adds only a single $\mathbb{G}_1$ point to the proof, a single pairing operation, and a single hashing operation per randomness.

One of the examples where UltraGroth has already been used is *Bionetta* – an R1CS-based zkML framework used by *Rarimo*. When implementing $\text{ReLU}(x) = \max\{0, x\}$ non-linearity in-circuit, in the vanilla Groth16 one needs to perform the full bit-decomposition of x , thus costing ≈ 254 constraints per call. As a result, the circuit size for the neural network is roughly $\mathcal{O}(n)$ with $n = 254L$ with L being the number of ReLU calls. Using UltraGroth, we reduced the number of constraints to the *sublinear complexity*: $\mathcal{O}(n/\log n)$.

0.1.1 Historical Notes

Before delving into specifics, we briefly mention the history of this protocol, which is quite curious. This blog is based on the original [Lev Soukhanov's post](#) in 2023. However, as we prepared the conference paper for Bionetta, we discovered that his construction is nothing but a generalization of the MIRAGE protocol [?], posted back in 2020! (in subsequent notation, MIRAGE is an UltraGroth protocol with $d = 1$). Even more surprisingly, it seems that independently of Lev Soukhanov's construction, Alex Ozdemir, Evan Laufer, and Dan Boneh introduced the MIRAGE+ protocol [?] in 2024, which was used for proving RAM computations correctness. This protocol, which is hard to believe, but also coincides with the Lev Soukhanov's construction (although I believe the former generalizes the construction a bit more rigorously). That said, to the best of my knowledge, Lev Soukhanov's construction is the first to generalize MIRAGE to the multiple-round interactive protocol.

All three papers (Bionetta included) proved completeness, soundness, and zero-knowledge of this construction, so the protocol that follows can be safely integrated into production systems. We, of course, drop formalities in this blog and recommend checking [?] for proof specifics in case someone is interested.

Finally, our personal opinion is that this construction is vastly underestimated and it is surprising that it is not yet used in production. Indeed, integrating lookup checks comes with a tiny cost of additional pairing which enables wrapping non-native protocols in Groth16 (e.g., zk-STARKs) *much* more effectively (up to a factor of $\times 20$).

Remark. The subsequent explanations will be quite technical, so if you want easier and more straightforward overview, you can additionally check out [our brief internal Distributed Lab presentation](#).

0.2 Lookup Checks

Here we restate the key fact that we explored in ??.

Theorem 0.1. Given two sequences of elements $\{t_i\}_{i \in [v]}$ and $\{z_i\}_{i \in [n]}$, the inclusion check $\{z_i\}_{i \in [n]} \subseteq \{t_i\}_{i \in [v]}$ is satisfied with overwhelming probability if and only if there exists the set of multiplicities $\{\mu_i\}_{i \in [v]}$ where $\mu_i = \#\{j \in [n] : z_j = t_i\}$ such that for randomly sampled $\gamma \leftarrow \mathbb{F}$:

$$\sum_{i \in [n]} \frac{1}{\gamma + z_i} = \sum_{i \in [v]} \frac{\mu_i}{\gamma + t_i}$$

More specifically, checking such equality at a random point from $\mathbb{F}$ results in the soundness error of up to $(n + v)/|\mathbb{F}|$, which becomes negligible for large $|\mathbb{F}|$.

Now, the right hand side of the equation, given that we have sampled the randomness $\gamma \leftarrow \mathbb{F}$, can be computed in v constraints, while the left hand side is computed in additional n constraints (recall that inversion costs a single constraint to prove). Again, since Groth16 is a non-interactive protocol in its essence (meaning, compared to PlonK or sumcheck-based protocols, it is not constructed by converting an interactive protocol to non-interactive), there is no yet effective way to sample γ (besides probably in-circuit hashing which is insanely costly). Yet, we do propose a way to sample γ by turning the Groth16 into the "interactive-like" proof system.

The proceeding *UltraGroth* construction is essentially devoted to sampling γ and passing it as a signal to the circuit soundly.

0.2.1 Some other implications

Sampling randomnesses overall enables numerous applications besides lookup checks. For instance, suppose you have matrices $A, B \in \mathbb{F}^{n \times n}$ and you want to write the circuit that computes their product: $C = AB$. The naive way to do that is to compute it by definition. However, such circuit costs $\mathcal{O}(n^3)$ constraints, which is a lot. Turns out that sampling randomnesses can reduce this significantly. Indeed, one can apply the *Freiveld's* protocol: first sample $\gamma \leftarrow \mathbb{F}^n$ and instead of checking $AB = C$, check whether matrices AB and C project γ to the same vector (this looks somewhat similar to Schwartz-Zippel lemma, but in the world of matrices):

$$(AB)\gamma = C\gamma \Leftrightarrow A(B\gamma) = C\gamma$$

Now notice that this can be implemented in approximately $3n^2$ constraints: one first multiplies $u \leftarrow B\gamma$, then multiplies Au and compares it to the product $C\gamma$. Note that matrix-vector product can be implemented in n^2 constraints, which is much more effective than matrix-matrix product with n^3 constraints.

However, currently, in Circom there is no way to pass γ as a signal to the circuit. Indeed, one can write `signal input gamma[n]` and mark it as public,

but how can the verifier be ensured that corresponding values in witness were indeed sampled uniformly randomly by the prover? So let's see how to fix it.

0.3 UltraGroth

0.4 Two-round Interactive R1CS

Suppose we split our R1CS into multiple rounds. Consider the case where we want to implement lookup checks. In such case, it would be nice if we could compile the following interactive protocol (IP) into the zk-SNARK for a relation $\mathcal{R}_{\text{QAP}}$:

- *Round 0:* The prover runs the circuit without imposing lookup checks. The prover sends resultant witness w_0 to verifier. Verifier responds with the random challenge $x_1 \leftarrow \mathbb{F}$.
- *Round 1:* The prover computes the witness w_1 that corresponds to the lookup check (that is, that verifies equality $\sum_i \frac{1}{x_1 + z_i} = \sum_i \frac{\mu_i}{x_1 + t_i}$) using the randomness x_1 provided by verifier. The prover sends w_1 and polynomial $h(X)$ according to $\mathcal{R}_{\text{QAP}}$.
- Finally, the verifier checks the same QAP equation $\ell(X)r(X) = o(X) + t_\Omega(X)h(X)$.

We would like to compile such IP into zk-SNARK by applying the Fiat-Shamir heuristic. However, here is the issue: x_1 must be derived by hashing the previous transcript, which is the public statement $\mathfrak{x}_0$ and witness w_0 . However, how can we hash the witness w_0 , given that the whole Groth16 proving system compiles the verification equation to just three points?

Now, suppose we partition the index set of public signals $\mathcal{I}_X$ into two parts (corresponding to each round): $\mathcal{I}_X^{(0)}$ and $\mathcal{I}_X^{(1)}$. Corresponding private signals are indexed by $\mathcal{I}_W^{(0)}$ and $\mathcal{I}_W^{(1)}$ which partition $\mathcal{I}_W$. In the example above, $\mathcal{I}_X^{(0)}$ corresponds to the regular public signals in relation $\mathcal{R}_{\text{QAP}}$ while $\mathcal{I}_X^{(1)}$ indexes the single solution witness element, corresponding to sampled randomness x_1 .

The idea is the following: we split the polynomial $c(X)$ (which was described in Groth16 construction above) into two pieces $c_0(X)$ and $c_1(X)$. The first piece corresponds to the first round indexed by $(\mathcal{I}_X^{(0)}, \mathcal{I}_W^{(0)})$ and is formed as follows: sample $r_0 \leftarrow \mathbb{F}$, add toxic parameter δ_0 , and compute the point $\pi_C^{(0)} \leftarrow g_1^{c_0(\tau)}$ where:

$$c_0(X) = \delta_0^{-1} \sum_{j \in \mathcal{I}_W^{(0)}} z_j f_j(X) + r_0 \delta$$

Notice that the point $\pi_C^{(0)}$ encodes all witness elements in Round 0. Thus, to sample randomness, we can hash prover parameters and this point: $x_1 \leftarrow \mathcal{H}(\text{pp}, \pi_C^{(0)})$. Then, points $\pi_A = g_1^{a(\tau)}$ and $\pi_B = g_1^{b(\tau)}$ are computed in the same way as in the

regular Groth16, while the third piece $\pi_C^{(1)} = g_1^{c_1(\tau)}$ is computed using:

$$c_1(X) = \delta^{-1} \left(\sum_{j \in \mathcal{I}_W^{(1)}} z_j f_j(X) + h(X) t_\Omega(X) \right) + a(X)s + b(X)r - r_0 \delta_0 - r s \delta$$

Essentially, such construction ensures that $\delta_0 c_0(X) + \delta c_1(X)$ gives exactly the polynomial $\delta c(X)$ in the original Groth16. This allows the prover to construct the proof as a tuple $\pi = (\pi_A, \pi_B, \pi_C^{(0)}, \pi_C^{(1)})$ and verifier to check it using:

$$e(\pi_A, \pi_B) = e(g_1^\alpha, g_2^\beta) \cdot e(g_1^{i(\tau)}, g_2^\gamma) \cdot e(\pi_C^{(0)}, g_2^{\delta_0}) \cdot e(\pi_C^{(1)}, g_2^\delta),$$

where $i(X) = \gamma^{-1} \sum_{i \in \mathcal{I}_X} z_i f_i(X)$ as before. As in Groth16, the term $e(g_1^\alpha, g_2^\beta)$ can be pre-computed, thus the cost of verification is 4 pairing operations.

Remark. The aforementioned protocol, as mentioned earlier, is essentially the MIRAGE protocol, and can be implemented as is. It is complete, sound, and zero-knowledge. In the subsequent section, we will generalize this protocol for more than 1 round.

0.5 Generalizing to multiple rounds

To generalize the aforementioned protocol, we define the $(d + 1)$ -round quadratic arithmetic program (or d QAP, for short) as follows:

$$\mathcal{R}_{d\text{QAP}} = \left\{ \begin{array}{l} \mathbb{x}_i = \{z_j\}_{j \in \mathcal{I}_X^{(i)}} \\ \mathbb{w}_i = \{z_j\}_{j \in \mathcal{I}_W^{(i)}} \\ \text{for } i \in [d + 1] \end{array} \middle| \begin{array}{l} \ell(X) \cdot r(X) = o(X) + t_\Omega(X)h(X) \\ \ell(X) = \sum_{i \in [n]} z_i \ell_i(X), \\ r(X) = \sum_{i \in [n]} z_i r_i(X), \\ o(X) = \sum_{i \in [n]} z_i o_i(X), \\ \text{for some } h(X) \in \mathbb{F}[X] \end{array} \right\},$$

where indices $\{\mathcal{I}_X^{(i)}\}_{i \in [d+1]}$ and $\{\mathcal{I}_W^{(i)}\}_{i \in [d+1]}$ partition $[n]$. In this notation, all $\{\mathbb{w}_i\}_{i \in [d+1]}$ are called *private witnesses*, $\mathbb{x}_0$ is a *public statement*, while $\{\mathbb{x}_i\}_{i \in [d+1] \setminus \{0\}}$ are *challenges* (that are going to be sampled by means of UltraGroth).

Since prover and verifier participate in the interactive protocol, the prover cannot compute the whole witness on his own without verifier's randomnesses. For that reason, we define *strategy* as the collection of functions $S = \{S_i\}_{i \in [d]}$, each of which computes the witness for the given round given previous witnesses and challenges and the current challenge, sampled by the verifier. In other words,

$$\mathbb{w}_i = S_i(\mathbb{x}_0, \dots, \mathbb{x}_i, \mathbb{w}_0, \dots, \mathbb{w}_{i-1})$$

Example 0.1. For instance, 0QAP represents the regular Groth16 construction, where strategy S_0 is a regular witness calculator: $\mathbb{w}_0 = S_0(\mathbb{x}_0)$. The protocol which involves lookup checks is a SNARK over 1QAP, where S_0 computes the

R1CS witness without lookup constraints, while S_1 , based on the sampled randomness(es) $\mathbb{x}_1$, imposes lookup check condition: $\mathbb{w}_1 = S_1(\mathbb{x}_0, \mathbb{x}_1, \mathbb{w}_0)$.

Now, we can describe the *UltraGroth* protocol over $\mathcal{R}_{\text{dQAP}}$.

Setup($1^\lambda, \mathcal{R}_{\text{dQAP}}$). As in previous sections, denote by $f_i(X) := \beta \ell_i(X) + \alpha r_i(X) + o_i(X)$ the random linear combination of ℓ_i, r_i, o_i . During the trusted setup, one generates the toxic waste $\alpha, \beta, \gamma, \{\delta_i\}_{i \in [d+1]}, \tau \leftarrow \$ \mathbb{F}$ and computes the following common parameters:

$$\begin{aligned} \text{pp} &= \left(g_1^\alpha, g_1^\beta, \{g_1^{\delta_i}\}_{i \in [d]}, g_2^\beta, g_2^\gamma, \{g_2^{\delta_i}\}_{i \in [d]}, \right. \\ &\quad \left. \left\{g_1^{\tau^i}, g_2^{\tau^i}, g_1^{\tau^i t(\tau)/\delta_d}\right\}_{i \in [n]}, \left\{g_1^{f_i(\tau)/\gamma}\right\}_{i \in \mathcal{I}_X}, \left\{g_1^{f_j(\tau)/\delta_i}\right\}_{(i,j) \in [d] \times \mathcal{I}_W^{(i)}} \right) \\ \text{vp} &= \left(g_1, g_2, g_2^\gamma, \{g_2^{\delta_i}\}_{i \in [d]}, g_T^{\alpha\beta}, \left\{g_1^{f_i(\tau)/\gamma}\right\}_{i \in \mathcal{I}_X} \right) \end{aligned}$$

Prove(pp, $\mathbb{x}_0, S$). The prover initializes the accumulator $a_0 := \mathcal{H}(\text{pp})$. For each round $i \in [d]$ she conducts the following steps:

- Samples $r_i \leftarrow \$ \mathbb{F}$
- Computes witness $\mathbb{w}_i \leftarrow S_i(\mathbb{x}_0, \dots, \mathbb{x}_i, \mathbb{w}_0, \dots, \mathbb{w}_{i-1})$.
- Computes $\pi_C^{(i)} \leftarrow g_1^{c_i(\tau)}$ with $c_i(X) := \delta_i^{-1} \sum_{j \in \mathcal{I}_W^{(i)}} z_j f_j(X) + r_i \delta_d$.
- Updates accumulator as $a_{i+1} \leftarrow \mathcal{H}(a_i, \pi_C^{(i)})$.
- If $i < d$, for each $j \in \mathcal{I}_X^{(i+1)}$ sets $z_j \leftarrow \mathcal{H}(a_{i+1}, g_1^j)$.

After finishing d rounds, the prover does the following:

- Computes polynomial $h(X)$ from QAP as in Groth16.
- Samples random $r, s \leftarrow \mathbb{F}$ and computes $\pi_A \leftarrow g_1^{a(\tau)}, \pi_B \leftarrow g_2^{b(\tau)}$ and the last proof piece $\pi_C^{(d)} \leftarrow g^{c_d(\tau)}$ where $a(X), b(X)$, and $c(X)$ are defined as follows:

$$\begin{aligned} a(X) &= \alpha + \sum_{i \in [n]} z_i \ell_i(X) + r \delta_d, \\ b(X) &= \beta + \sum_{i \in [n]} z_i r_i(X) + s \delta_d \\ c_d(X) &= \delta_d^{-1} \left(\sum_{i \in \mathcal{I}_W^{(d)}} z_i f_i(X) + h(X) t_\Omega(X) \right) + a(X)s + b(X)r - \sum_{i \in [d]} r_i \delta_i - r s \delta_d \end{aligned}$$

- **Outputs** proof $\pi = (\pi_A, \pi_B, \pi_C^{(0)}, \dots, \pi_C^{(d)}) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1^{d+1}$.

Verify(vp, $\mathbb{x}_0, \pi$). The verifier recomputes all the challenges $\{\mathbb{x}_i\}_{i \in [d+1] \setminus \{0\}}$ and verifies that corresponding public signals are correct. Then, she checks:

$$e(\pi_A, \pi_B) = e(g_1^\alpha, g_2^\beta) \cdot e(g_1^{i(\tau)}, g_2^\gamma) \cdot \prod_{i \in [d+1]} e(\pi_C^{(i)}, g_2^{\delta_i})$$

0.6 Efficiency Analysis

Finally, we specify the efficiency of UltraGroth.

- The prover’s work is essentially the same as in Groth16: if the total witness size is n , the prover needs to conduct $\mathcal{O}(n \cdot E_1 + n \cdot E_2)$ operations with a couple of additional hashes (formally, $\sum_{i \in [d+1] \setminus \{0\}} |\mathbb{X}_i|$ hashes, which is a negligible amount in practice). Note however, that due to such R1CS splitting, the circuit’s size n is drastically decreased: in many cases it becomes sublinear in the original size.
- Verifier’s work is $(d + 3)E + \mathcal{O}(l \cdot E_1)$.
- Proof size is 1 $\mathbb{G}_2$ point and $(d + 2)$ $\mathbb{G}_1$ points.

0.7 Practical Implementation and Further Steps

Currently, we have implemented the UltraGroth as a part of the Bionetta project for $d = 1$ in the following repository: <https://github.com/rarimo/ultragroth>. Also, we have a custom *SnarkJS* fork that implements smart contract generation and witness verify and export operations. But the current implementation is vastly inflexible and currently primarily works in conjunction with *Bionetta’s witness generator* only. This implementation takes the regular Circom-generated `.wtns` and `.sym` files, parses them, figures out which witness element corresponds to the randomness, and during the proving procedure fills in this spot using the protocol above.

We look forward to completing the implementation that could be easily integrated into other’s projects. One nice way to implement that would be to add a special keyword to the Circom compiler. For instance, it would be nice if we could write `random signal r[N]` that fills in the vector `r` with random values, although such task is surely challenging.

0.8 Acknowledgements

Of course, without Lev Soukhanov this section would not exist. I also thank Artem Sdobnov, Vitalii Volovyk, Yevhenii Sekhin, and Illia Dovgopoly for the UltraGroth implementation for Bionetta.

0.9 Summary

In case you implement complex circuits involving non-native operations (e.g., wrapping the verification of some ZK protocol), it is definitely worth checking out the UltraGroth protocol. With the tiny verification overhead, one gets an immense boost to the proving procedure. In particular, it makes client-side ZK much more practical and realistic without sacrificing verification efficiency.